

# DOMAIN NAME SYSTEM (DNS)

Reti di Calcolatori (Modulo 2)

# Introduction

- ▶ IP addresses (e.g., IPv4 o IPv6) used to identify network resources
- ▶ Humans are more comfortable with names
  - ▶ Prefer pronounceable names rather than numeric addresses
- ▶ Need of a service and scheme that maps high level names on IP addresses and vice versa
- ▶ Domain Name System (DNS) provides the mapping between names and resources

# Names For Computers: History

- ▶ Earliest computer systems forced users to use numeric addresses
- ▶ Timesharing systems allow to use name for physical and abstract objects
- ▶ The same applies to networking environments
- ▶ Hardware addresses *mapped to* IP addresses *mapped to* symbolic names

ARP/RARP



DNS



# Names For Computers: History

- ▶ The form of the names is important
  - ▶ Historically few names necessary
  - ▶ Today millions of machine connected to the Internet
- ▶ 1980: few dozens of machine connected
- ▶ Mid 1986: 3,100 hosts and 6,500 aliases
- ▶ 1990: 137,000 hosts with names
- ▶ 2000: more than 60 millions
- ▶ 2013: around one billion

# Names For Computers

- ▶ DNS applies to the Global Internet
- ▶ DNS provides an example of large scale client-server architecture
- ▶ Two possibilities
  - ▶ Flat namespace
  - ▶ Hierarchical namespace

# Flat and hierarchical names

- ▶ Flat name identifies an end-point
  - ▶ No hierarchical structure (e.g., “Davide”, “Milano”, ecc.).
- ▶ A hierarchical name shows how to reach the end-point
  - ▶ `www.tin.it`
  - ▶ `62.211.65.12`

# Flat and hierarchical names

- ▶ ARPANET (flat): 1970
  - ▶ Mapping name-address in a single file host.txt managed by SRI-NIC (Stanford Research Institute - Network Information Center)
  - ▶ Sent each night to each host in the network
  - ▶ Problems
    - ▶ High traffic overhead
    - ▶ High probability of name conflict
- ▶ Hierarchical DNS: 1983
  - ▶ By Paul Mockapetris (RFC 1034 and 1035)
  - ▶ Modified, updated and improved by subsequent RFC documents

# Flat namespaces

- ▶ Each name consisted of a sequence of characters
- ▶ The Network Information Center (NIC) responsible for management of namespaces
- ▶ Advantages
  - ▶ Convenient and short

# Flat namespaces

## ▶ Disadvantages

- ▶ Cannot generalize to large sets of machines
- ▶ Names drawn from a single set of identifiers
  - ▶ The more the sites are, the more the number of potential conflicts
- ▶ Centralized approach based on a single authority responsible for the whole workload of adding new names *end SPOF*
- ▶ Mapping changes rapidly -> difficult to maintain a correct copy of the mapping at each site

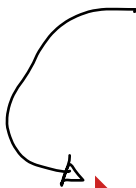
# Hierarchical namespaces

- ▶ How can a naming system be suitable for a large-scale scenario?

# Hierarchical namespaces

- ▶ How can a naming system be suitable for a large-scale scenario?
- ▶ Decentralization of naming mechanisms
- ▶ Authorities delegated for parts of the namespaces

# Naming Hierarchy

- ▶ Partitioning in a way that supports efficient mapping and autonomous control
  - ▶ Two possibilities
    - ▶ According to network topology
    - ▶ By organizational structure (independent of physical networks)
- ▶ Internet uses the latter
- 

# Internet Hierarchy

- ▶ *In a TCP/IP internet, hierarchical machine names are assigned according to the structure of organizations that obtain authority for parts of the namespace, not necessarily according to the structure of the physical network interconnections*

# Internet Domain Names

- ▶ Flexible hierarchy
  - ▶ Universal naming scheme (same everywhere)
  - ▶ Each organization determines internal naming structure
- ▶ Mechanism known as *Domain Name System (DNS)*
  - ▶ Implements a machine name hierarchy
- ▶ Name assigned to a computer known as *domain name*

# Domain Name System

- ▶ Two roles
  - ▶ It specifies the name syntax, and rules for delegating authorities
  - ▶ It specifies the implementation of a distributed computing system that maps names to addresses

# Domain Name Syntax

- ▶ Set of *labels separated by a delimiter character (period)*
- ▶ Example
  - ▶ *cs . purdue . edu*
- ▶ Three labels: *cs, purdue, and edu*
- ▶ String *purdue . edu* **is also a domain**
- ▶ Top-level domain is *edu*

# Domain Name Hierarchies

- ▶ Domain name standard only specifies an **abstract hierarchical namespace**
  - ▶ **Arbitrary values** for labels
- ▶ Most users use the hierarchical labels by the official Internet domain system
  - ▶ Comprehensive and flexible
  - ▶ Suitable for TCP/IP with no need to change names

# Domain Name Hierarchies

- ▶ Two different naming hierarchies
  - ▶ Geographic scheme divides the universe of machines by countries
    - ▶ laf.in.us
    - ▶ crema.cr.it
  - ▶ Organizational scheme groups organizations by type
    - ▶ xinu.cs.purdue.edu
  - ▶ Mix
    - ▶ tremotino.dti.unimi.it

# Original Top-Level Domains

| Domain Name         | Assigned To                          |
|---------------------|--------------------------------------|
| com                 | Commercial organizations             |
| edu                 | Educational institutions (4-year)    |
| gov                 | Government institutions              |
| mil                 | Military groups                      |
| net                 | Major network support centers        |
| org                 | Organizations other than those above |
| arpa                | Temporary ARPANET domain (obsolete)  |
| int                 | International organizations          |
| <i>country code</i> | Each country (geographic scheme)     |

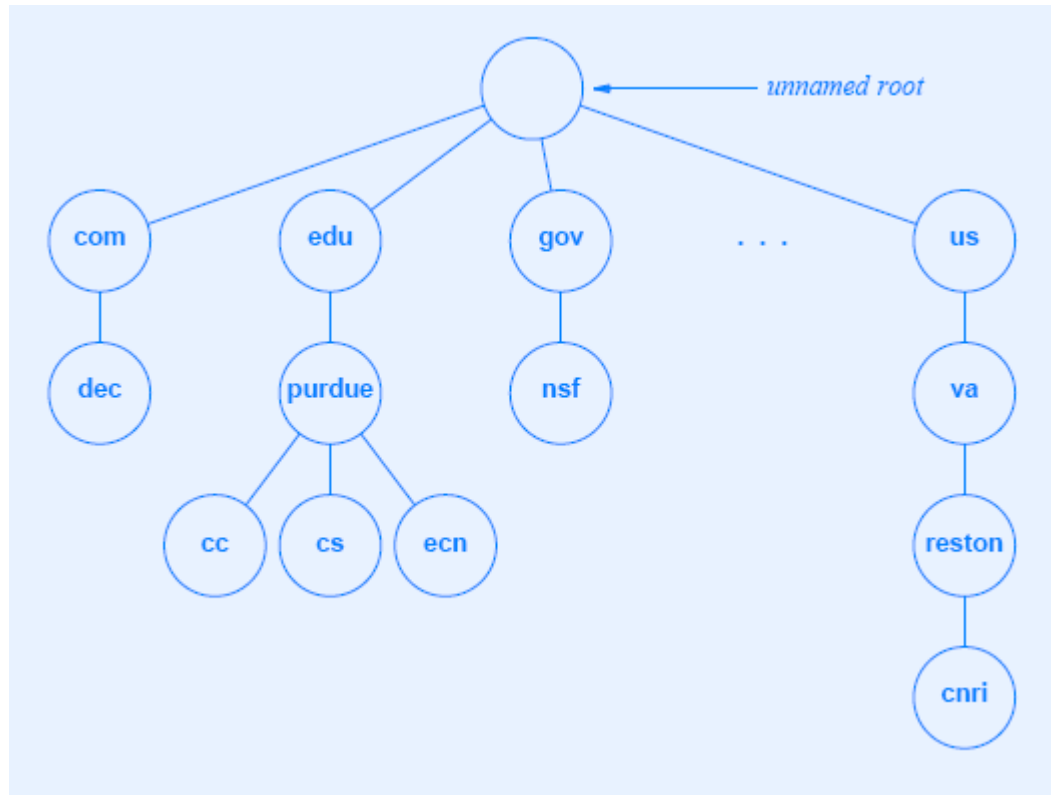
- ▶ Meaning assigned to each
- ▶ Three domains considered generic
  - ▶ .com
  - ▶ .net
  - ▶ .org

# New Top-Level Domains (first round)

| Domain Name | Assigned To                                      |
|-------------|--------------------------------------------------|
| aero        | Air-Transport Industry                           |
| biz         | Businesses                                       |
| coop        | Non-Profit Cooperatives                          |
| info        | Unrestricted                                     |
| museum      | Museums                                          |
| name        | Individuals                                      |
| pro         | Professionals (accountants, lawyers, physicians) |

- ▶ Defined in 2000-01
- ▶ Other rounds till 23 October 2013: start largest-ever expansion of the Domain Name System (DNS)
- ▶ From the 22 gTLDs (e.g., .COM, .BIZ, .ORG) to more than 1400 new possibilities
- ▶ Registry listing: <https://www.icann.org/resources/pages/listing-2012-02-25-en>

# Illustration Of Part Of The DNS Tree



# Authority For Names

- ▶ Authority delegated down the tree
- ▶ Example
  - ▶ Purdue University registers under top level domain *.edu* and receives authority for domain *purdue . edu*
  - ▶ Computer Science Department at Purdue registers with the Purdue authority, and becomes the authority for *cs . purdue . edu*
  - ▶ Owner of a lab in the CS Department registers with the departmental authority, and becomes the authority for *xinu . cs . purdue . edu*

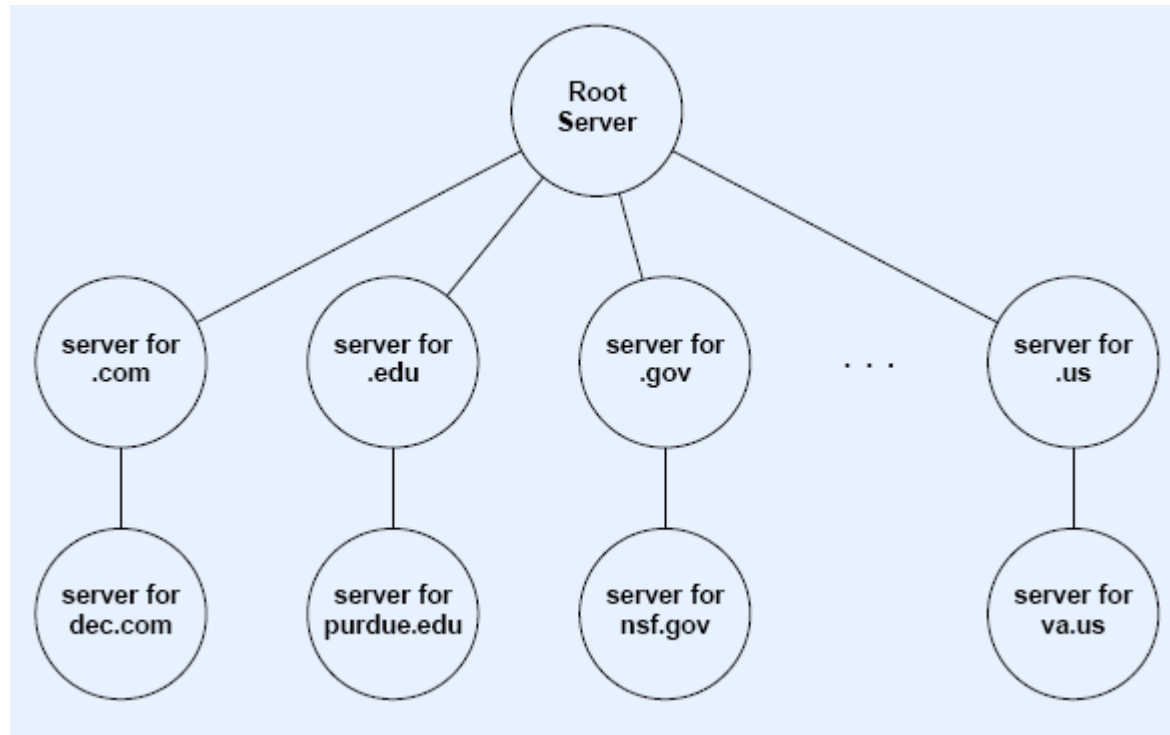
# Mapping Domain Names To Addresses

- ▶ DNS uses a set of on-line servers
- ▶ Servers arranged in tree
- ▶ A given server can handle entire subtrees
  - ▶ Example: ISP manages domain names for its clients (including corporations)

# Terminology

- ▶ DNS server known as *name server*
  - ▶ Supplies name-to-address translation
- ▶ DNS client software known as *resolver*
  - ▶ Use one or more name server to do the translation

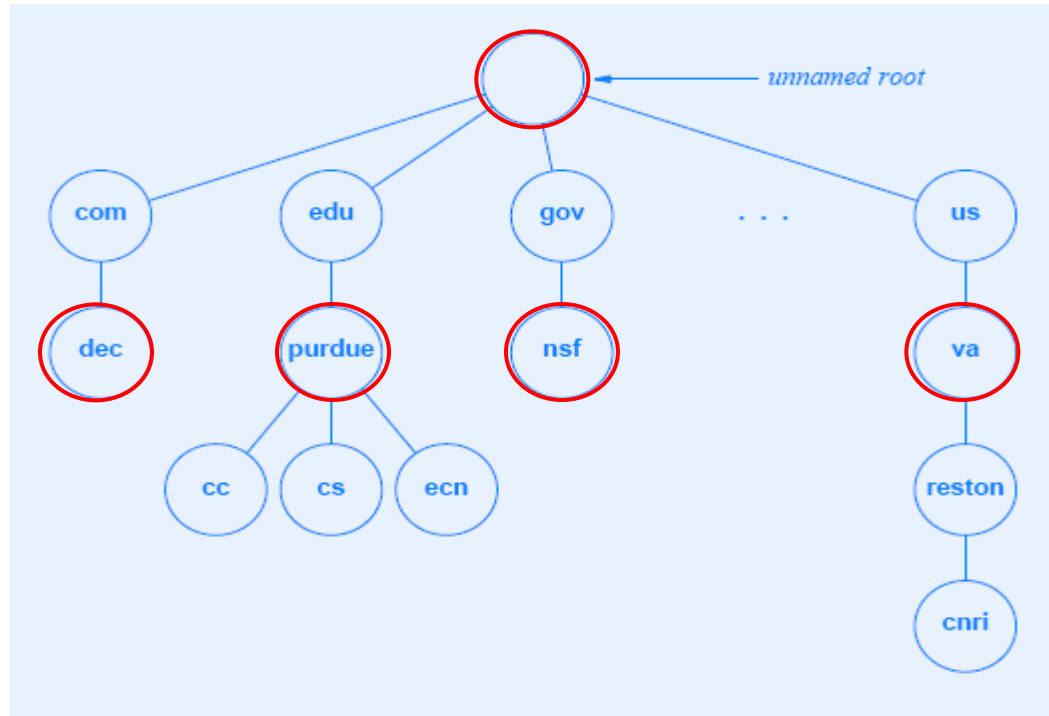
# Illustration of Topology Among DNS Servers



# In Practice

- ▶ Single server can handle multiple levels of the naming tree
- ▶ Example: root server handles all top-level domains
- ▶ Links in the conceptual tree are not physical connections
  - ▶ The server may be located everywhere

# In Practice



- ▶ Red circle represents mapping between a naming hierarchy and a realistic organization of servers

# DNS Database

- ▶ Record has (name, class, type)
- ▶ DNS partitions the set of names by classes
  - ▶ The domain system allows to specify the protocol family
  - ▶ Class is IN for internet applications
- ▶ Type specifies the type of object and entry (e.g., computer, email exchanger)
- ▶ Consequence
  - ▶ *A given name may map to more than one item in the domain system. The client specifies the type of object desired when resolving a name, and the server returns objects of that type*

# DNS Database

- ▶ The syntax of a name does not determine the type of object or the class of protocol suite
  - ▶ Gwen.purdue.edu indistinguishable from cs.purdue.edu
- ▶ *One cannot distinguish the names of subdomains from the names of individual objects or the type of an object using only the domain name syntax*

# Domain Name Resolution

- ▶ Conceptually, must search from root of tree downward
- ▶ In practice
  - ▶ Every name server knows location of a root server
  - ▶ Only contact root if no mapping is found by name server in the sub-domains for which it is an authority
  - ▶ Lookup always starts with local server first (host can learn address of DNS server from DHCP)

↳  
Richiesta richiesta  
Buona possibilità di ricevere un  
mapping → Server mantiene cache

# Domain Name Resolution

- ▶ Domain name query contains
  - ▶ Name to be resolved
  - ▶ Class of the name (e.g., IN)
  - ▶ Type of answer desired (e.g, MX)
  - ▶ Code that specifies the type of interaction
    - ▶ Complete translation or recursive resolution
    - ▶ Non-recursive resolution or iterative resolution

# Domain Name Resolution

- ▶ When a DNS receives a query
  - ▶ Lookup always starts with local server first (host can learn address of DNS server from DHCP)
  - ▶ If it cannot translate the name completely, it checks the type of interaction requested by the client
    - ▶ Complete translation or recursive resolution: the server contacts a server that can do the translation
    - ▶ Non-recursive resolution or iterative resolution: the server generates a reply with the name of the server able to do the translation

# Efficient Translation

- ▶ Facts
  - ▶ Name-to-address bindings change infrequently
  - ▶ User is likely to repeat same lookup
- ▶ Starting from the root servers has some disadvantages
  - ▶ Most lookups refer to local names
  - ▶ Root servers overloaded
  - ▶ Failure of topmost-level block local requests

# Efficient Translation

- ▶ To increase efficiency (2-step)
  - ▶ Initial contact begins with local name server
    - ▶ Every server caches answers (owner specifies cache timeout)
  - ▶ If the server is not able to translate the request is sent to another server
    - ▶ Root server
    - ▶ Direct ancestor

# Caching: Server

- ▶ Cost of lookup for nonlocal names high

*↳ Per come tutto lo stack ISO-OSI*

- ▶ Name caching to optimize search cost

① ▶ The server first checks if it is authoritative

② ▶ Then it checks the cache

③ ▶ Finally it asks other servers

- ▶ Cached information are marked nonauthoritative

*↳ Risposta è stata data da un altro server*

# Caching: Server

- ▶ Caching works well
  - ▶ Changes are infrequent
- ▶ Each entry has an expiration time
  - ▶ When expired, entry in the cache is discarded
  - ▶ The server can directly contact the authoritative source

# Caching: Client

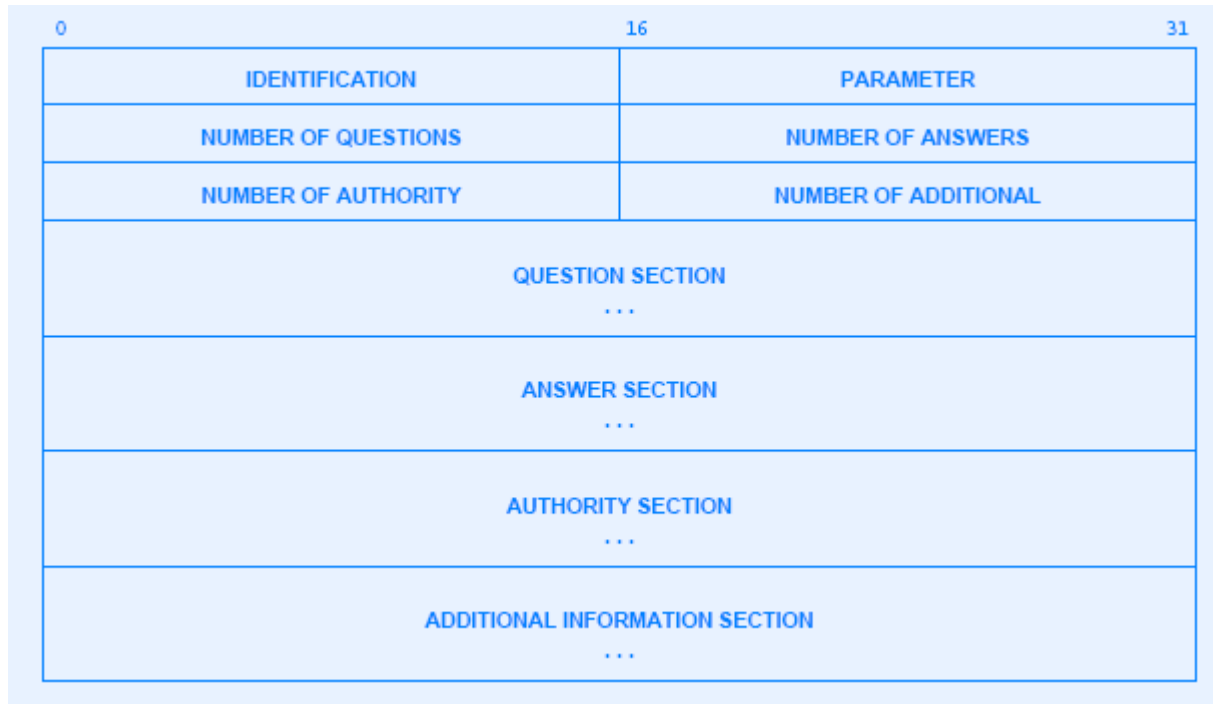
- ▶ Caching works well also for the client
- ▶ The client downloads the database from the local DNS
- ▶ The cache must be updated periodically
- ▶ Several advantages: fast, robust (against server failures), scalable (less computational overhead)

# Message Flow

1. User invokes an application and pass a name of a machine
2. The application passes the domain name to the local resolver (local DB and cache)
3. If the name cannot be resolved, it is sent to the local server
4. Either the server returns the IP address (recursive or authoritative or cache) or other name servers that can resolve the address



# Domain Server Message Format



# Domain Server Message Format

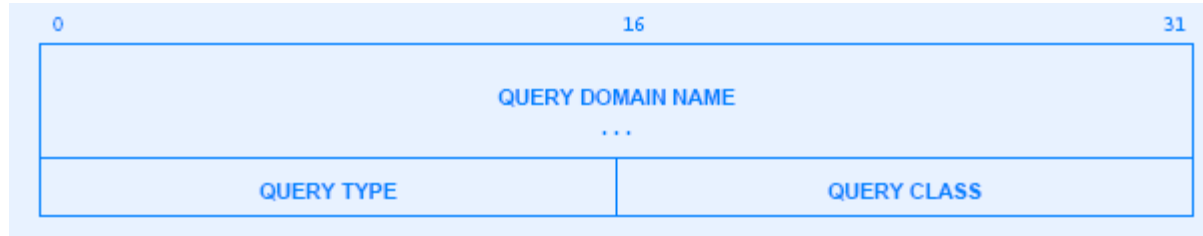
- ▶ IDENTIFICATION: used to match response to query
- ▶ PARAMETER: requested operation and response code  $\rightarrow f_{e23}$
- ▶ NUMBER OF \*: count of entries in the corresponding SECTION
- ▶ QUESTION SECTION: queries
- ▶ ANSWER, AUTHORITY, ADDITIONAL SECTIONS: resource records

# Parameter Bits

| Bit of PARAMETER field | Meaning                                                                                                                                                                                                                                                                                     |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0                      | Operation:<br>0 Query<br>1 Response                                                                                                                                                                                                                                                         |
| 1-4                    | Query Type:<br>0 Standard<br>1 Inverse<br>2 Server status request<br>3 Completion (now obsolete)<br>4 Notify<br>5 Update                                                                                                                                                                    |
| 5                      | Set if answer authoritative                                                                                                                                                                                                                                                                 |
| 6                      | Set if message truncated                                                                                                                                                                                                                                                                    |
| 7                      | Set if recursion desired                                                                                                                                                                                                                                                                    |
| 8                      | Set if recursion available                                                                                                                                                                                                                                                                  |
| 9                      | Set if data is authenticated                                                                                                                                                                                                                                                                |
| 10                     | Set if checking is disabled                                                                                                                                                                                                                                                                 |
| 11                     | Reserved                                                                                                                                                                                                                                                                                    |
| 12-15                  | Response Type:<br>0 No error<br>1 Format error in query<br>2 Server failure<br>3 Name does not exist<br>5 Refused<br>6 Name exists when it should not<br>7 RR set exists<br>8 RR set that should exist does not<br>9 Server not authoritative for the zone<br>10 Name not contained in zone |

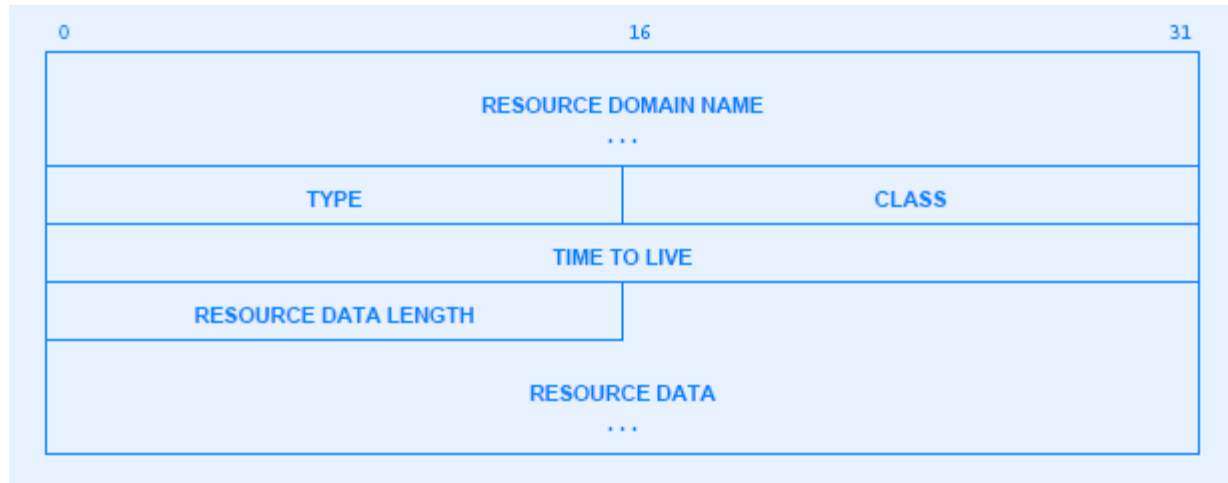


# Format of Question Section



- ▶ A triple
  - ▶ Domain name (e.g., dti.unimi.it)
  - ▶ A type (e.g., NS)
  - ▶ A class (e.g., IN)

# Format of Resource Records



- ▶ Resource data field contains the mapping
- ▶ Also used to store answers

# Object Types That DNS Supports

| Type  | Meaning            | Contents                                                                             |
|-------|--------------------|--------------------------------------------------------------------------------------|
| A     | Host Address       | 32-bit IP address                                                                    |
| CNAME | Canonical Name     | Canonical domain name for an alias                                                   |
| HINFO | CPU & OS           | Name of CPU and operating system                                                     |
| MINFO | Mailbox info       | Information about a mailbox or mail list                                             |
| MX    | Mail Exchanger     | 16-bit preference and name of host that acts as mail exchanger for the domain        |
| NS    | Name Server        | Name of authoritative server for domain                                              |
| PTR   | Pointer            | Domain name (like a symbolic link)                                                   |
| SOA   | Start of Authority | Multiple fields that specify which parts of the naming hierarchy a server implements |
| TXT   | Arbitrary text     | Uninterpreted string of ASCII text                                                   |
| AAAA  | Host Address       | 128-bit IPv6 address                                                                 |

# Resource Record (RR) Content

- ▶ DNS used to translate domain name in mail exchanger address or IP address or ...
  - ▶ RR used to model the knowledge of a name server and all the mapping
- ▶ Most of translation of type A
- ▶ MX translation is useful, **why?**

# Resource Record (RR) Content

- ▶ DNS used to translate domain name in mail exchanger address or IP address or ...
  - ▶ RR used to model the knowledge of a name server and all the mapping
- ▶ Most of translation of type A
- ▶ MX translation is useful
  - ▶ Take the domain part of an email address and issue a MX query type
  - ▶ A set of resource records are returned with domain name
  - ▶ Domain name to IP address mapping can also be inserted in the ADDITIONAL INFORMATION SECTION

# Obtaining Authority for Domains

- ▶ Having authority for a second level domain means support Internet standards
  - ▶ Support protocol standards
  - ▶ Know each server for the subdomains and a root server
  - ▶ Support concurrent activities and requests
  - ▶ Provide replication
    - ▶ No single point of failure

# Obtaining Authority for Domains

- ▶ A single server may manage different parts of the naming hierarchy
- ▶ *Zone of authority*: a subtree of names managed by a given name server
- ▶ For each zone exists a file with one or more resource records

↳ Ogni zona del DB contiene un mapping

# Abbreviation of Domain Names

- ▶ DNS only recognizes full domain names
- ▶ Client software allows abbreviation

# Example of Domain Name Abbreviation

- ▶ Client configured with suffix list
  - ▶ **. cs . purdue . edu**
  - ▶ **. cc . purdue . edu**
  - ▶ **. purdue . edu**
  - ▶ *null*
- ▶ User enters abbreviation *xinu*
- ▶ Client tries the following in order
  - ▶ **xinu. cs . purdue . edu**
  - ▶ **xinu. cc . purdue . edu**
  - ▶ **xinu. purdue . edu**
  - ▶ **xinu**

# The Point About Abbreviation

- ▶ *The Domain Name System only maps full domain names into addresses; abbreviations are not part of the Domain Name System itself, but are introduced by client software to make local names convenient for users*
- ▶ *Also used to restrict application programs to local names*

# Pointer Query

- ▶ Special case of inverse mapping
  - ▶ IP address -> domain name
  - ▶ Special form of query
  - ▶ Special domain ARPA
- ▶ Convert IP address to domain name
- ▶ Trick: write IP address as a string and lookup as a name

# Why Pointer Query?

- ▶ Network diagnostic (similar to ping)
- ▶ A technique to check for spam messages
  - ▶ Domain name in PTR record matches the mail server name
- ▶ Forward Confirmed reverse DNS (FCrDNS), weak form of authentication
- ▶ SSH sessions refused if requests come from a client of an unknown domain

# Example of Pointer Query

- ▶ Start with dotted decimal address such as  
*aaa . bbb . ccc . ddd*
- ▶ Rearrange dotted decimal representation as a string  
*ddd . ccc . bbb . aaa . in-addr . arpa*
- ▶ Lookup using a *pointer query type*

# Summary

- ▶ Domain Name System provides mapping from pronounceable names to IP addresses
- ▶ Domain names are hierarchical; top-level domains are dictated by a central authority
- ▶ Organizations can choose how to structure their domain names
- ▶ DNS uses on-line servers to answer queries
- ▶ Lookup begins with local server, which caches entries

